

Углубленное машинное обучение и искусственный интеллект в Python

Патласов Дмитрий Александрович, ПГНИУ
dmitriypatlasov@gmail.com

Общая картина: Агент Альфред

Познакомьтесь с Альфредом.
Альфред - агент.



Представьте, что Альфред получает команду, например: «Альфред, я бы хотел кофе, пожалуйста»



Поскольку Альфред понимает естественный язык, он быстро понимает нашу просьбу.

Перед выполнением заказа Альфред занимается рассуждениями и планированием, определяя, какие действия и инструменты ему понадобятся:

Пойти на кухню

Воспользоваться кофеваркой

Заварите кофе

Принесите кофе обратно

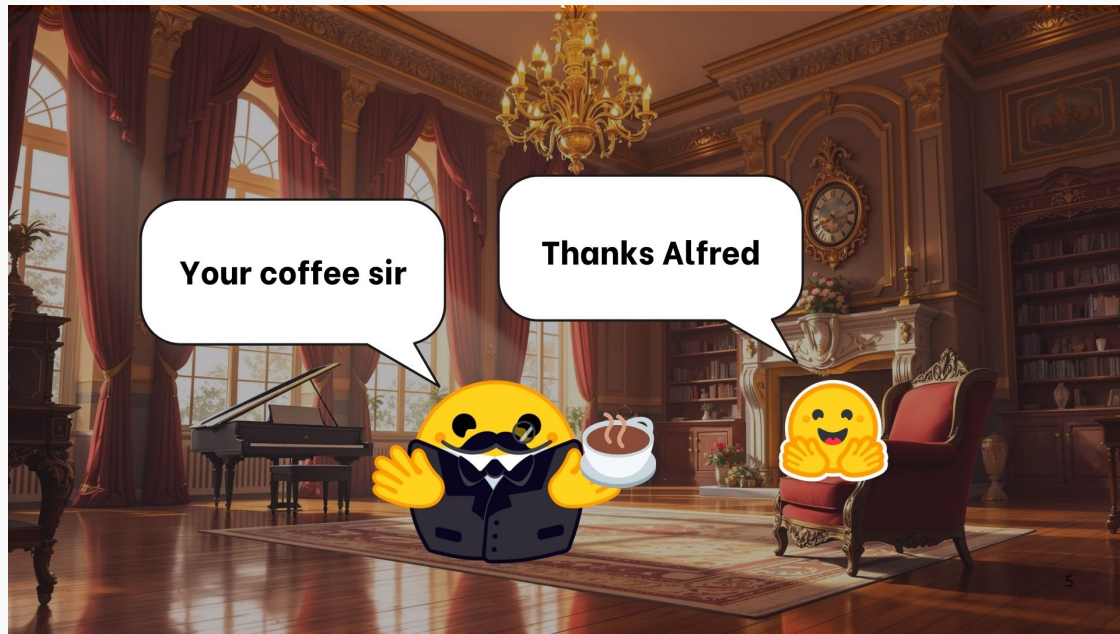


Когда у него есть план, он должен действовать. Чтобы выполнить свой план, он может использовать инструменты из списка инструментов, о которых он знает.

В данном случае, чтобы приготовить кофе, он использует кофеварку. Он активирует кофеварку, чтобы сварить кофе.



Наконец Альфред приносит нам
свежесваренный кофе.



Именно это и есть агент: ИИ модель, способная рассуждать, планировать и взаимодействовать со своим окружением.

Мы называем его агентом, потому что он обладает агентностью, то есть способностью взаимодействовать с окружающей средой.



Агент - это система, использующая модель искусственного интеллекта для взаимодействия с окружающей средой с целью достижения определенной пользователем цели. Он сочетает в себе рассуждения, планирование и выполнение действий (часто с помощью внешних инструментов) для выполнения задач.

1. Мозг (модель ИИ).

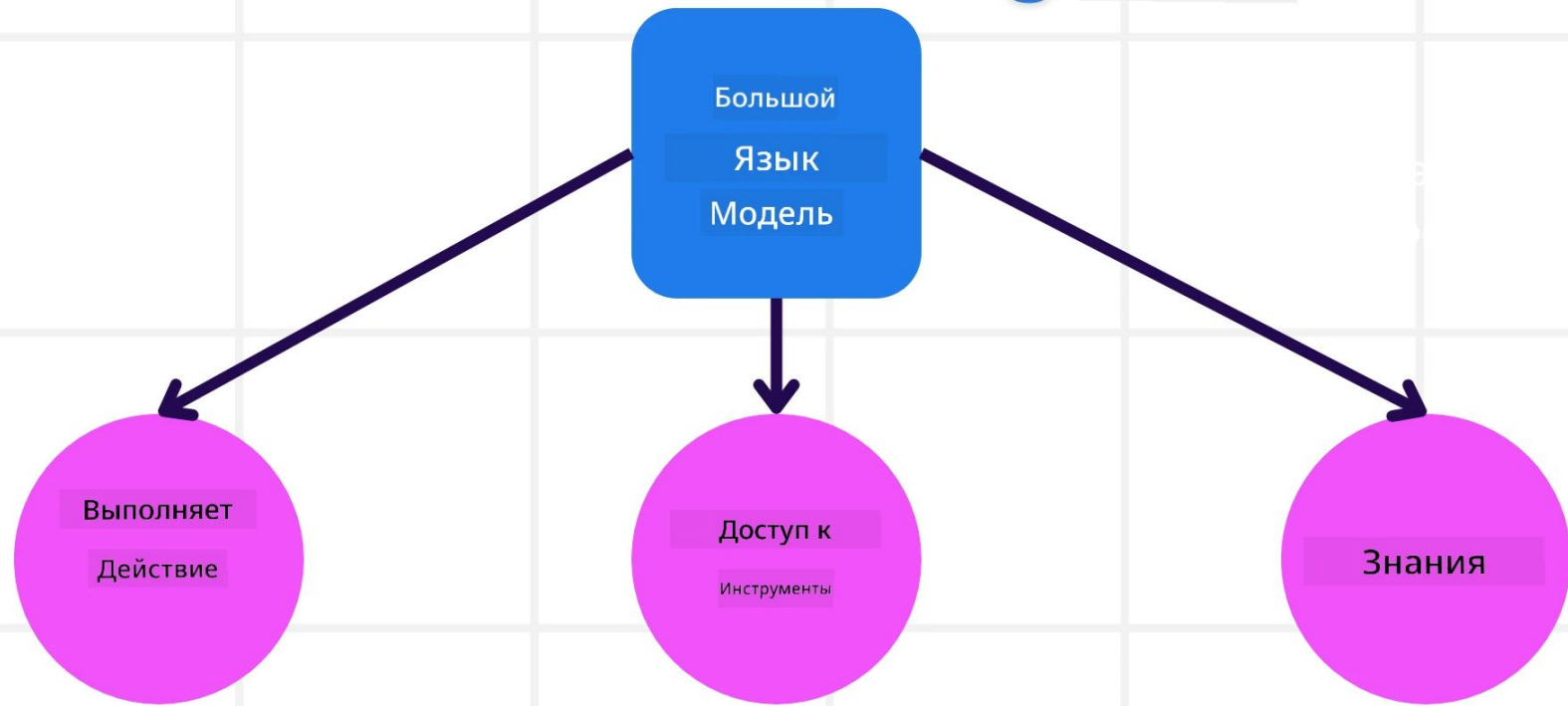
Именно здесь происходит все мышление. Модель ИИ занимается рассуждениями и планированием. Она решает, какие действия предпринять в зависимости от ситуации.

2. Тело (возможности и инструменты).

Эта часть представляет собой все, что агент способен делать.

Набор возможных действий зависит от того, чем наделен агент. Например, поскольку у человека нет крыльев, он не может выполнять действие «летать», но может выполнять такие действия**, как «ходить», «бегать», «прыгать», «хватать» и так далее.

Что такое AI-агенты?



aka.ms/ai-agents-beginners

Система — AI-агент — это не одна вещь. Это набор частей, работающих вместе

- Окружающая среда — пространство, в котором агент действует. Для агента по бронированию путешествий это будет сама платформа бронирования.
- Датчики — как агент читает текущее состояние окружающей среды. Наш агент может проверять доступность отелей или цены на авиабилеты.
- Исполнители (актуаторы) — как агент совершает действия. Агент бронирования может забронировать номер, отправить подтверждение или отменить бронь.
- Большие языковые модели — агенты существовали до появления LLM, но именно LLM делают современные агенты такими мощными. Они понимают естественный язык, анализируют контекст и могут преобразовывать нечеткий запрос пользователя в конкретный план действий.
- Выполнение действий — без системы агента LLM генерирует лишь текст. В системе агента LLM может выполнять шаги — искать в базе данных, вызывать API, отправлять сообщение.
- Доступ к инструментам — какие инструменты агент может использовать, зависит от (1) окружающей среды, в которой он работает, и (2) того, что разработчик решил ему предоставить. Агент бронирования может искать авиарейсы, но не редактировать записи клиентов — всё зависит от связи.
- Память и знания — у агентов может быть кратковременная память (текущий разговор) и долговременная (база данных клиентов, прошлые взаимодействия). Агент может «помнить», что вы предпочитаете место у окна.

Разные типы AI-агентов

Тип агента	Что он делает	Пример агента по бронированию
Простые рефлексные агенты	Следует жестким правилам — без памяти и планирования.	Видит письмо с жалобой → пересылает в службу поддержки. Вот и всё.
Рефлексные агенты на основе модели	Хранит внутреннюю модель мира и обновляет её при изменениях.	Отслеживает исторические цены на авиабилеты и отмечает маршруты с резким подорожанием.
Целевые агенты	Имеет цель и планирует, как достичь её шаг за шагом.	Бронирует всю поездку (билеты, машину, отель) с учетом вашего текущего местоположения и пункта назначения.
Агенты, основанные на полезности	Ищет не просто решение, а <i>лучшее</i> , взвешивая компромиссы.	Балансирует стоимость и удобство, чтобы подобрать поездку, максимально подходящую под ваши предпочтения.
Обучающиеся агенты	Улучшается со временем, обучаясь на обратной связи.	Корректирует рекомендации по бронированию на основе отзывов после поездки.
Иерархические агенты	Высокоуровневый агент разбивает работу на подзадачи и делегирует нижестоящим агентам.	Запрос “отмена поездки” делится на: отмена авиабилетов, отеля, аренды машины — каждая задача поручена отдельному агенту.
Мультиагентные системы (MAS)	Несколько независимых агентов работают вместе (или конкурируют).	Кооперация: разные агенты отвечают за отели, авиабилеты и развлечения. Конкуренция: несколько агентов борются за бронирование номеров по лучшей цене.

Какие модели ИИ используются для агентов?

Наиболее распространенной моделью ИИ, используемой в агентах, является LLM (большая языковая модель), которая принимает на вход Текст и выводит Текст.

Известными примерами являются GPT4 от OpenAI, LLama от Meta, Gemini от Google и т. д. Эти модели были обучены на огромном количестве текстов и способны к обобщению.

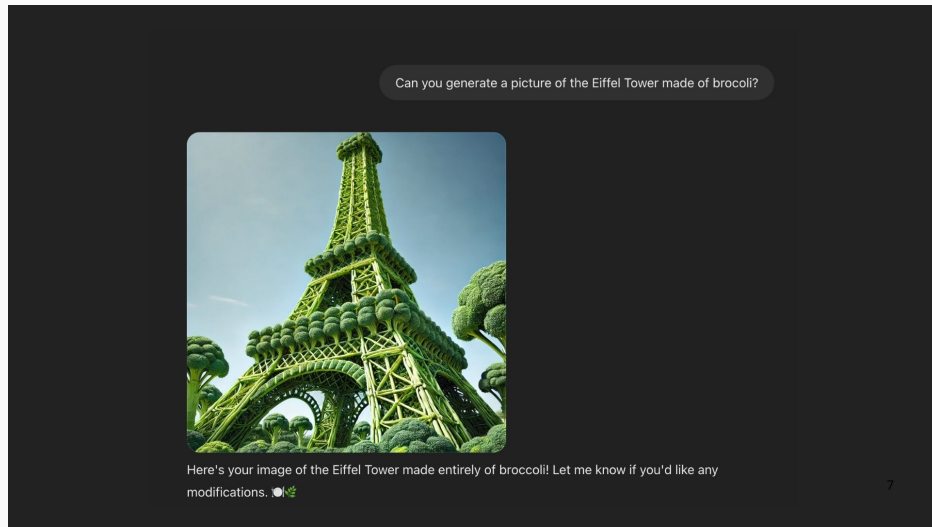
Также можно использовать модели, принимающие другие входные данные, в качестве основной модели агента. Например, Vision Language Model (VLM), которая похожа на LLM, но также понимает изображения в качестве входных данных.

Как ИИ воздействует на окружающую среду?

LLM - замечательные модели, но они могут генерировать только текст.

Однако если вы попросите известное чат-приложение, например HuggingChat или ChatGPT, сгенерировать изображение, они смогут! Как такое возможно?

Ответ заключается в том, что разработчики HuggingChat, ChatGPT и подобных приложений реализовали дополнительный функционал (так называемые инструменты (tools)), которые LLM может использовать для создания изображений.



Какой тип задач может выполнять Агент?

Агент может выполнять любые задачи, которые мы реализуем с помощью Инструментов для выполнения Действий.

Например, если я напишу агента, который будет действовать как мой личный помощник (как Siri) на моем компьютере, и попрошу его «отправить электронное письмо моему менеджеру с просьбой отложить сегодняшнюю встречу», я могу дать ему код для отправки электронных писем. Это будет новый инструмент, который агент сможет использовать всякий раз, когда ему понадобится отправить письмо. Мы можем написать его на языке Python:

```
def send_message_to(recipient, message):  
  
    """Используется для отправки электронного сообщения получателю"""  
  
    ...
```

LLM, как мы увидим, будет генерировать код для запуска инструмента, когда это необходимо, и таким образом выполнять поставленную задачу.

```
send_message_to("Менеджер", "Мы можем отложить сегодняшнюю встречу?")
```

Пример 1: Виртуальные персональные ассистенты

Виртуальные помощники, такие как Siri, Alexa или Google Assistant, работают как агенты, когда взаимодействуют от имени пользователей в их цифровом окружении.

Они принимают запросы пользователей, анализируют контекст, извлекают информацию из баз данных, дают ответы или инициируют действия (например, устанавливают напоминания, отправляют сообщения или управляют смарт-устройствами).

Пример 2: Чат-боты для обслуживания клиентов

Многие компании используют чат-боты в качестве агентов, которые взаимодействуют с клиентами на естественном языке.

Эти агенты могут отвечать на вопросы, направлять пользователей по шагам устранения неисправностей, открывать проблемы во внутренних базах данных или даже завершать транзакции.

Их заранее определенные цели могут включать повышение удовлетворенности пользователей, сокращение времени ожидания или увеличение коэффициента конверсии продаж. Взаимодействуя непосредственно с клиентами, обучаясь в ходе диалога и адаптируя свои ответы с течением времени, они демонстрируют основные принципы работы агента в действии.

Пример 3: ИИ неигрового персонажа в видеоигре

Вкратце, агент - это система, которая использует модель ИИ (обычно LLM) в качестве основного механизма рассуждений, чтобы:

Вместо того чтобы следовать жесткому дереву поведения, они могут реагировать контекстно, адаптироваться к взаимодействию с игроком и генерировать более тонкие диалоги. Такая гибкость помогает создавать более реалистичные и увлекательные персонажи, которые развиваются вместе с действиями игрока.

Вкратце, агент - это система, которая использует ИИ модель (обычно LLM) в качестве основного механизма рассуждений, чтобы:

- Понимать естественный язык: Интерпретировать и осмысленно отвечать на человеческие инструкции.
- Рассуждать и планировать: Анализировать информацию, принимать решения и разрабатывать стратегии для решения проблем.
- Взаимодействовать с окружающей средой: Собирать информацию, предпринимать действия и наблюдать за их результатами.

Понимание AI Агентов через цикл Мысль - Действие - Наблюдение

Агенты работают в непрерывном цикле: думать (Мысль) → действовать (Действие) и наблюдать (Наблюдение).

Давайте разложим эти действия на составляющие:

1. Мысль: LLM-часть Агента решает, каким должен быть следующий шаг.
2. Действие: Агент выполняет действие, вызывая инструменты с соответствующими аргументами.
3. Наблюдение: Модель размышляет над ответом, полученным от инструмента.

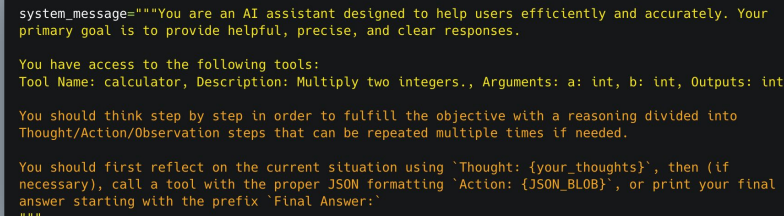
Цикл “Мысль-Действие-Наблюдение”

Все три компонента работают вместе в непрерывном цикле. Если воспользоваться аналогией из программирования, агент использует цикл `while`: цикл продолжается до тех пор, пока не будет выполнена поставленная перед агентом задача.

Во многих фреймворках Агентов правила и рекомендации встраиваются непосредственно в системную подсказку, гарантируя, что каждый цикл будет следовать определенной логике.

Здесь мы видим, что в системном сообщении мы определили :

1. Поведение Агента.
2. Инструменты, к которым имеет доступ наш Агент, как мы описали в предыдущем разделе.
3. Цикл Мысль-Действие-Наблюдение, который мы заложили в инструкции LLM.



```
system_message="""You are an AI assistant designed to help users efficiently and accurately. Your
primary goal is to provide helpful, precise, and clear responses.

You have access to the following tools:
Tool Name: calculator, Description: Multiply two integers., Arguments: a: int, b: int, Outputs: int

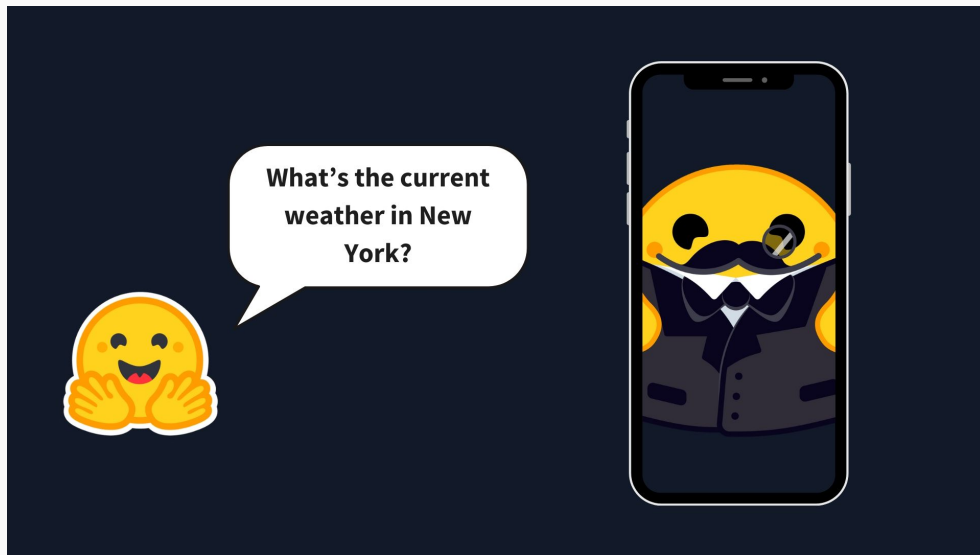
You should think step by step in order to fulfill the objective with a reasoning divided into
Thought/Action/Observation steps that can be repeated multiple times if needed.

You should first reflect on the current situation using 'Thought: {your_thoughts}', then (if
necessary), call a tool with the proper JSON formatting 'Action: {JSON_BLOB}', or print your final
answer starting with the prefix 'Final Answer:'
"""
```

Альфред, Агент сообщающий погоду

Пользователь спрашивает
Альфреда: «Какая сегодня
погода в Нью-Йорке?».

Задача Альфреда - ответить на
этот запрос, используя
инструмент API погоды.



Мысль

Внутренние рассуждения:

Получив запрос, внутренний диалог Альфреда может быть таким:

“Пользователю нужна текущая информация о погоде в Нью-Йорке. У меня есть доступ к инструменту, который получает данные о погоде. Сначала мне нужно обратиться к API погоды, чтобы получить актуальную информацию.”

Этот шаг показывает, что агент разбивает проблему на этапы: сначала собирает необходимые данные.



Действие

Использование инструмента:

Основываясь на своих рассуждениях и на том факте, что Альфред знает об инструменте `get_weather`, Альфред подготавливает команду в формате JSON, которая вызывает инструмент API погоды. Например, его первым действием может быть:

Мысль: Мне нужно проверить текущую погоду в Нью-Йорке.

```
{  
  "action": "get_weather",  
  "action_input": {"location": "New York"}  
}
```

Здесь действие четко указывает, какой инструмент следует вызвать (например, `get_weather`) и какой параметр передать (`"location": "New York"`).



Наблюдение

Обратная связь от окружающей среды:.

После вызова инструмента Альфред получает наблюдение. Это могут быть необработанные данные о погоде из API, например:

“Текущая погода в Нью-Йорке: частично облачно, 15°C, влажность 60%.”



Обновленная мысль

Рефлексия:

Получив данные наблюдения,
Альфред обновляет свои
внутренние рассуждения:

“Теперь, когда у меня есть данные
о погоде в Нью-Йорке, я могу
подготовить ответ для
пользователя.”



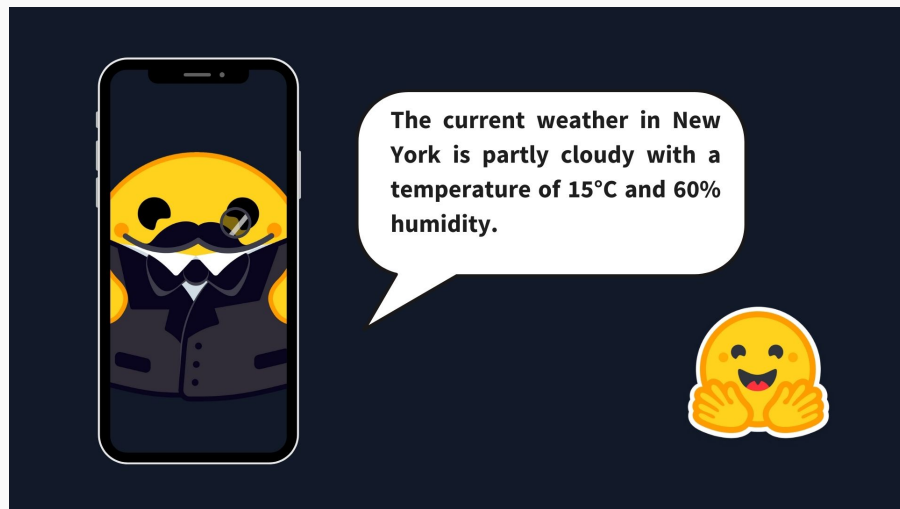
Финальное Действие

Затем Алфред генерирует окончательный ответ, отформатированный так, как мы ему сказали:

Мысль: У меня есть данные о погоде. Текущая погода в Нью-Йорке частично облачная, температура 15 °C и влажность 60 %».

Окончательный ответ : Текущая погода в Нью-Йорке частично облачно с температурой 15 °C и влажностью 60 %.

Это заключительное действие возвращает ответ пользователю, закрывая цикл.



Что мы видим в этом примере

1. Агенты проходят цикл до тех пор, пока цель не будет достигнута:

Процесс Альфреда цикличен. Он начинает с мысли, затем действует, вызывая инструмент, и, наконец, наблюдает за результатом. Если бы наблюдение показало ошибку или неполноту данных, Альфред мог бы снова войти в цикл, чтобы скорректировать свой подход.

2. Интеграция инструментов:

Способность вызывать инструменты (например, API погоды) позволяет Альфреду выходить за пределы статических знаний и получать данные в реальном времени, что является важным аспектом многих AI Агентов.

3. Динамическая адаптация:

Каждый цикл позволяет агенту включать свежую информацию (наблюдения) в свои рассуждения (мысли), гарантируя, что окончательный ответ будет хорошо обоснованным и точным.

Этот пример демонстрирует основную концепцию цикла ReAct (концепцию, которую мы будем развивать в следующем разделе): взаимодействие мыслей, действий и наблюдений позволяет AI агентам решать сложные задачи итеративно.

Понимая и применяя эти принципы, вы сможете разрабатывать агентов, которые не только рассуждают о своих задачах, но и эффективно используют внешние инструменты для их выполнения, при этом постоянно совершенствуя свои действия на основе обратной связи от окружающей среды.

Агенты и фреймворки

1) OpenClaw

→ персональный AI-агент, работает на твоих устройствах и подключается к 50+ платформам

2) AutoGPT

→ платформа для создания и запуска автономных AI-агентов

3) LangChain

→ база для создания LLM-приложений и агентных систем

4) OpenCode

→ AI-кодинг агент прямо в терминале

16 Must-Know AI GitHub Repos in 2026 DailyDoseofDS.com

1 OpenClaw  Personal AI agent that runs on your devices and connects to 50+ messaging platforms	2 AutoGPT  Platform for building, deploying, and running autonomous AI agents	3 Hugging Face Transformers  The model framework for state-of-the-art ML across text, vision, audio, and multimodal	4 Ollama  Run powerful LLMs locally on your hardware with a single command
5 LangChain  The foundational framework for building agents and LLM-powered applications	6 Open WebUI  Self-hosted, offline-capable ChatGPT alternative with built-in RAG and plugin system	7 ComfyUI  Node-based visual workflow builder for AI image and video generation applications	8 Sim  Open-source drag-and-drop workflow builder for creating and deploying AI agent pipelines
9 Opik  Open-source platform to trace, evaluate, and monitor LLM apps and agentic workflows	10 Firecrawl  Turn any website into LLM-ready markdown or structured data	11 Airweave  Open-source context retrieval layer that syncs 50+ data sources for AI agents	12 vLLM  High-throughput, memory-efficient LLM serving engine for production deployments
13 Unsloth  Fine-tune and run open models 2x faster with 70% less memory	14 OpenPipe ART  Train multi-step AI agents for real-world tasks using RL	15 OpenCode  Open-source, provider-agnostic AI coding agent built for the terminal	16 Chandra OCR (by Datalab)  State-of-the-art OCR model for complex tables, forms, handwriting, and 90+ languages

Модели и локальный запуск

5) Hugging Face Transformers

→ главный фреймворк для SOTA моделей (текст, vision, audio)

6) Ollama

→ запуск LLM локально одной командой

7) vLLM

→ быстрый и эффективный inference-движок для продакшена

8) Unsloth

→ ускоренный fine-tuning (x2 быстрее, -70% памяти)

16 Must-Know AI GitHub Repos in 2026 DailyDoseofDS.com

1 OpenClaw



Personal AI agent that runs on your devices and connects to 50+ messaging platforms

2 AutoGPT



Platform for building, deploying, and running autonomous AI agents

3 Hugging Face Transformers



The model framework for state-of-the-art ML across text, vision, audio, and multimodal

4 Ollama



Run powerful LLMs locally on your hardware with a single command

5 LangChain



The foundational framework for building agents and LLM-powered applications

6 Open WebUI



Self-hosted, offline-capable ChatGPT alternative with built-in RAG and plugin system

7 ComfyUI



Node-based visual workflow builder for AI image and video generation applications

8 Sim



Open-source drag-and-drop workflow builder for creating and deploying AI agent pipelines

9 Opik



Open-source platform to trace, evaluate, and monitor LLM apps and agentic workflows

10 Firecrawl



Turn any website into LLM-ready markdown or structured data

11 Airweave



Open-source context retrieval layer that syncs 50+ data sources for AI agents

12 vLLM



High-throughput, memory-efficient LLM serving engine for production deployments

13 Unsloth



Fine-tune and run open models 2x faster with 70% less memory

14 OpenPipe ART



Train multi-step AI agents for real-world tasks using RL

15 OpenCode



Open-source, provider-agnostic AI coding agent built for the terminal

16 Chandra OCR (by Datalab)



State-of-the-art OCR model for complex tables, forms, handwriting, and 90+ languages

Инструменты и UI

9) Open WebUI

→ self-hosted альтернатива ChatGPT с RAG и плагинами

10) ComfyUI

→ визуальный workflow для генерации изображений и видео

11) Sim

→ drag-and-drop конструктор AI пайплайнов

16 Must-Know AI GitHub Repos in 2026 DailyDoseofDS.com

1 **OpenClaw**



Personal AI agent that runs on your devices and connects to 50+ messaging platforms

2 **AutoGPT**



Platform for building, deploying, and running autonomous AI agents

3 **Hugging Face Transformers**



The model framework for state-of-the-art ML across text, vision, audio, and multimodal

4 **Ollama**



Run powerful LLMs locally on your hardware with a single command

5 **LangChain**



The foundational framework for building agents and LLM-powered applications

6 **Open WebUI**



Self-hosted, offline-capable ChatGPT alternative with built-in RAG and plugin system

7 **ComfyUI**



Node-based visual workflow builder for AI image and video generation applications

8 **Sim**



Open-source drag-and-drop workflow builder for creating and deploying AI agent pipelines

9 **Opik**



Open-source platform to trace, evaluate, and monitor LLM apps and agentic workflows

10 **Firecrawl**



Turn any website into LLM-ready markdown or structured data

11 **Airweave**



Open-source context retrieval layer that syncs 50+ data sources for AI agents

12 **vLLM**



High-throughput, memory-efficient LLM serving engine for production deployments

13 **Unsloth**



Fine-tune and run open models 2x faster with 70% less memory

14 **OpenPipe ART**



Train multi-step AI agents for real-world tasks using RL

15 **OpenCode**



Open-source, provider-agnostic AI coding agent built for the terminal

16 **Chandra OCR**
(by Datalab)



State-of-the-art OCR model for complex tables, forms, handwriting, and 90+ languages

Данные, RAG и инфраструктура

12) Firecrawl

→ превращает любой сайт в markdown/структурированные данные

13) Airweave

→ слой контекста: синхронизация 50+ источников для агентов

14) Opik

→ мониторинг, трейсинг и оценка LLM-приложений

16 Must-Know AI GitHub Repos in 2026 DailyDoseofDS.com

1 OpenClaw  Personal AI agent that runs on your devices and connects to 50+ messaging platforms	2 AutoGPT  Platform for building, deploying, and running autonomous AI agents	3 Hugging Face Transformers  The model framework for state-of-the-art ML across text, vision, audio, and multimodal	4 Ollama  Run powerful LLMs locally on your hardware with a single command
5 LangChain  The foundational framework for building agents and LLM-powered applications	6 Open WebUI  Self-hosted, offline-capable ChatGPT alternative with built-in RAG and plugin system	7 ComfyUI  Node-based visual workflow builder for AI image and video generation applications	8 Sim  Open-source drag-and-drop workflow builder for creating and deploying AI agent pipelines
9 Opik  Open-source platform to trace, evaluate, and monitor LLM apps and agentic workflows	10 Firecrawl  Turn any website into LLM-ready markdown or structured data	11 Airweave  Open-source context retrieval layer that syncs 50+ data sources for AI agents	12 vLLM  High-throughput, memory-efficient LLM serving engine for production deployments
13 Unsloth  Fine-tune and run open models 2x faster with 70% less memory	14 OpenPipe ART  Train multi-step AI agents for real-world tasks using RL	15 OpenCode  Open-source, provider-agnostic AI coding agent built for the terminal	16 Chandra OCR (by Datalab)  State-of-the-art OCR model for complex tables, forms, handwriting, and 90+ languages

Специализированные модели

15) OpenPipe ART

→ обучение multi-step AI-агентов через reinforcement learning

16) Chandra OCR

→ SOTA OCR для таблиц, форм, рукописного текста и 90+ языков

16 Must-Know AI GitHub Repos in 2026 DailyDoseofDS.com

1 **OpenClaw**



Personal AI agent that runs on your devices and connects to 50+ messaging platforms

2 **AutoGPT**



Platform for building, deploying, and running autonomous AI agents

3 **Hugging Face Transformers**



The model framework for state-of-the-art ML across text, vision, audio, and multimodal

4 **Ollama**



Run powerful LLMs locally on your hardware with a single command

5 **LangChain**



The foundational framework for building agents and LLM-powered applications

6 **Open WebUI**



Self-hosted, offline-capable ChatGPT alternative with built-in RAG and plugin system

7 **ComfyUI**



Node-based visual workflow builder for AI image and video generation applications

8 **Sim**



Open-source drag-and-drop workflow builder for creating and deploying AI agent pipelines

9 **Opik**



Open-source platform to trace, evaluate, and monitor LLM apps and agentic workflows

10 **Firecrawl**



Turn any website into LLM-ready markdown or structured data

11 **Airweave**



Open-source context retrieval layer that syncs 50+ data sources for AI agents

12 **vLLM**



High-throughput, memory-efficient LLM serving engine for production deployments

13 **Unsloth**



Fine-tune and run open models 2x faster with 70% less memory

14 **OpenPipe ART**



Train multi-step AI agents for real-world tasks using RL

15 **OpenCode**



Open-source, provider-agnostic AI coding agent built for the terminal

16 **Chandra OCR (by Datalab)**



State-of-the-art OCR model for complex tables, forms, handwriting, and 90+ languages



Смотря какой agent

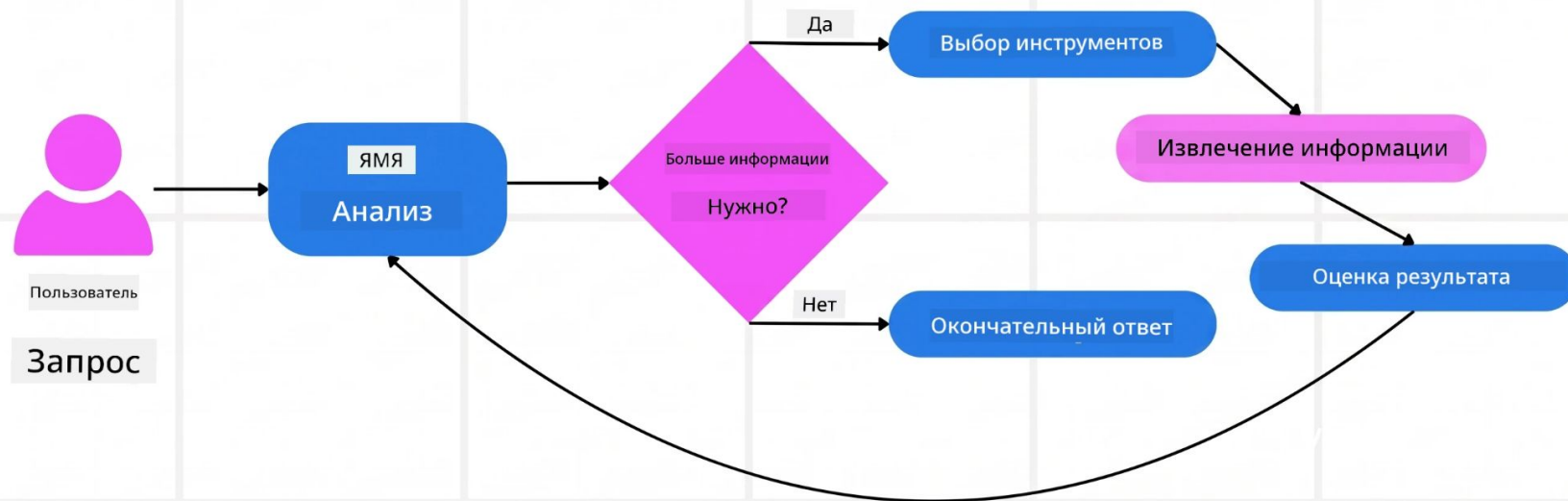


Смотря сколько tokens

Agentic RAG

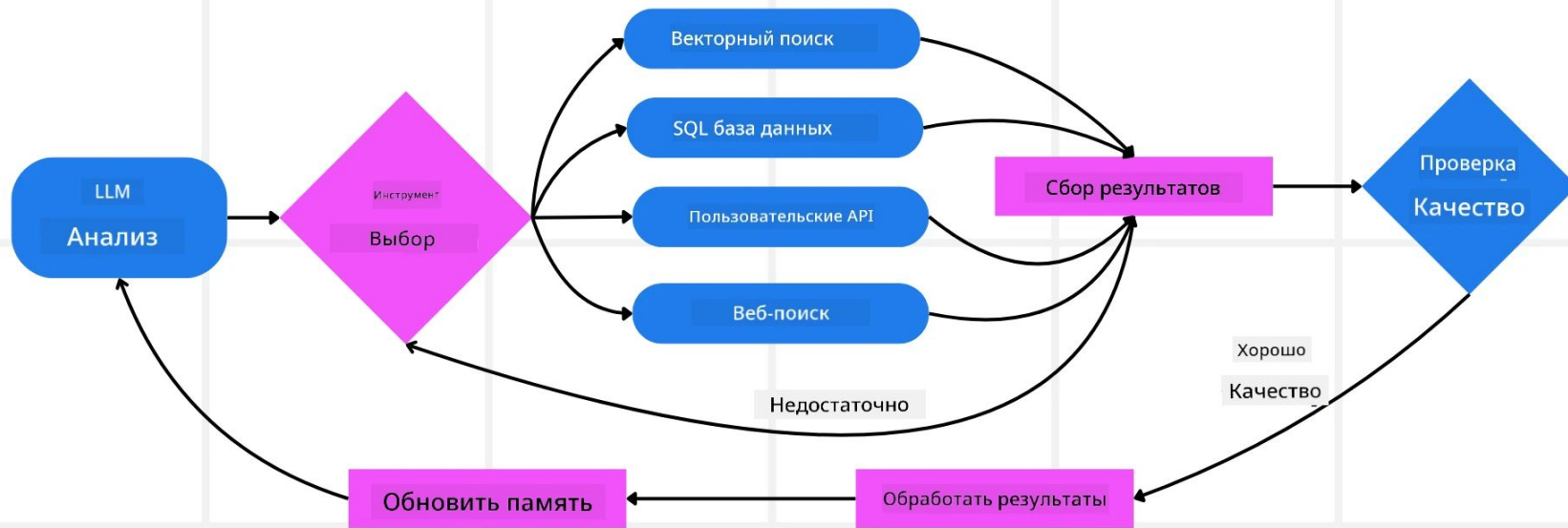
Agentic Retrieval-Augmented Generation (Agentic RAG) — это новая парадигма в разработке ИИ, где большие языковые модели (LLM) не только извлекают информацию из внешних источников данных, но и автономно планируют свои следующие шаги. В отличие от статичных схем «извлечение-потом-чтение» или тщательно продуманных последовательностей подсказок, Agentic RAG включает цикл итеративных вызовов LLM, прерываемых вызовами инструментов или функций и структурированными выводами. На каждом шаге система оценивает полученные результаты, решает, нужно ли уточнить запросы, при необходимости вызывает дополнительные инструменты и продолжает этот цикл, пока не достигнет удовлетворительного решения.

Основной цикл Agentic RAG



aka.ms/ai-agents-beginners

Интеграция инструментов Agentic RAG

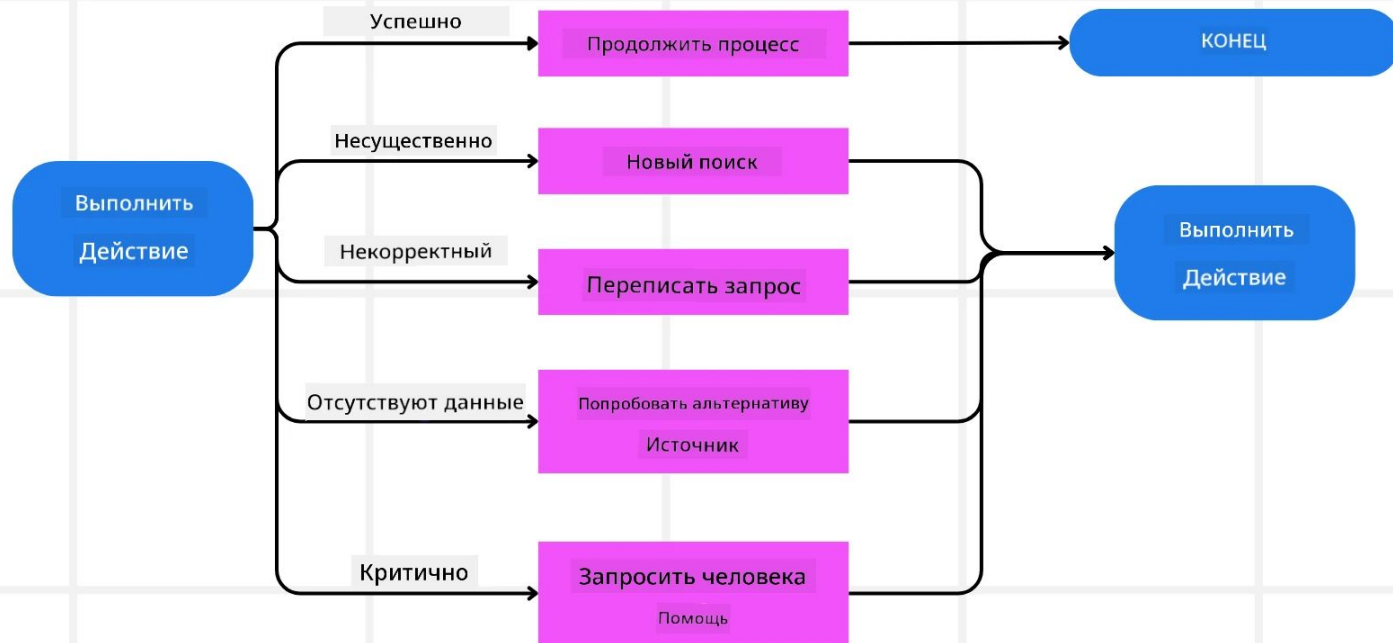


aka.ms/ai-agents-beginners

Агентная система опирается на циклический паттерн взаимодействия:

- Первоначальный вызов: Цель пользователя (т.е. запрос пользователя) представляется LLM.
- Вызов инструмента: Если модель обнаруживает отсутствующую информацию или неоднозначные инструкции, она выбирает инструмент или метод извлечения — например, запрос к векторной базе данных (например, гибридный поиск Azure AI Search по приватным данным) или структурированный SQL-вызов — чтобы собрать больше контекста.
- Оценка и уточнение: После изучения возвращённых данных модель решает, достаточно ли информация. Если нет, она уточняет запрос, пробует другой инструмент или изменяет подход.
- Повторять до удовлетворения: Этот цикл продолжается до тех пор, пока модель не определит, что у неё достаточно ясности и доказательств для предоставления окончательного, хорошо обоснованного ответа.
- Память и состояние: Поскольку система сохраняет состояние и память между шагами, она может вспоминать предыдущие попытки и их результаты, избегая повторяющихся циклов и принимая более информированные решения по мере продвижения.

Агентное самокорректирование RAG



aka.ms/ai-agents-beginners